

Informe de compilación — RedEconómica v1.0.0

Fecha de generación: 28 de agosto de 2026

1. Veredicto

COMPILACIÓN NO VERIFICADA

No se ejecutó `./gradlew assembleDebug` y no se entrega APK.

No es una omisión: el entorno donde se generó el proyecto no puede compilar Android. Lo que sí se pudo verificar está en el apartado 4, y está separado de lo que no, en el apartado 3.

2. Por qué

El proyecto se generó en un contenedor Linux en la nube que:

- **no tiene Android SDK instalado** (`$ANDROID_HOME` vacío, sin `platforms/android-34`, sin `build-tools`);
- **no tiene acceso de red a los repositorios de artefactos**. Comprobado con `curl` durante la preparación:

Destino	Resultado
<code>https://dl.google.com/android/repository/repository2-3.xml</code>	conexión rechazada (código 000)
<code>https://services.gradle.org/distributions/</code>	conexión rechazada (código 000)
<code>https://repo.maven.apache.org/maven2/</code>	conexión rechazada (código 000)
<code>https://registry.npmjs.org/</code>	200 (solo npm y PyPI están permitidos)

Sin esos tres destinos, Gradle no puede descargar el Android Gradle Plugin, el compilador de Kotlin, Compose ni Room, así que ninguna tarea del ciclo `clean → test → lint → assemble` llega a ejecutarse.

Se intentó ejecutar `gradle` (8.14.3, presente en el contenedor) y falló en la resolución de plugins con:

```
Plugin [id: 'com.android.application', version: '8.5.2'] was not found in any of the following sources
```

3. Tareas NO ejecutadas

Tarea	Estado	Motivo
<code>./gradlew clean</code>	No ejecutada	Sin resolución de plugins
<code>./gradlew testDebugUnitTest</code>	No ejecutada	Sin dependencias de prueba
<code>./gradlew lintDebug</code>	No ejecutada	Sin Android SDK
<code>./gradlew assembleDebug</code>	No ejecutada	Sin Android SDK
Firma / APK	No generado	Consecuencia de lo anterior
Prueba instrumentada	No ejecutada	Requiere dispositivo o emulador

Resultados de pruebas: desconocidos. Las 136 pruebas JVM están escritas y revisadas, pero **no se han ejecutado**. Este informe no afirma que pasen.

APK: no existe. Por tanto no hay tamaño ni SHA-256 de APK que informar.

4. Lo que Sí se verificó de verdad

Estas comprobaciones se ejecutaron realmente en el entorno, con herramientas que sí estaban disponibles (Python 3, SQLite).

4.1 Solubilidad y coherencia de los 42 desafíos

`tools/verificar_escenarios.py` reimplementa de forma independiente las reglas de `TradeEngine`, `SpecializationEngine`, `CooperationEngine`, `ScarcityEngine` y `OpportunityCostEngine`, y las aplica al contenido semilla.

Ejecutado. Resultado: sin problemas. Salida resumida:

```
== Intercambios ==          (8 escenarios, todas las soluciones aceptadas)
== ¿Aceptarías este trato? == (4 escenarios, respuesta esperada = veredicto del motor)
== Especialización ==      s01 s03 s07 s08 s09 s23 s28 s34 s41 → ≥1 reparto válido cada uno
== Cooperación ==          s19 s20(3) s22 s29 s38 → ≥1 reparto válido; cooperar > trabajar por separado
== Escasez ==              s02 s13 s14 s27 s32 s37 → urgencias altas siempre cubribles
== Decisiones ==           s15(4) s16(4) s31(5) s36(4) combinaciones asequibles, ninguna con todo
== Costo de oportunidad == s17=3 s18=1 s33=2
Todos los escenarios comprobados tienen solución y son coherentes.
```

Comprobación extra incluida en ese script: en el escenario s12, el trato con Lía **debe** ser rechazado con motivo `PERDERIA_LO_QUE_NECESITA` (es la lección del desafío). El motor lo rechaza con ese motivo exacto.

4.2 SQL cargado en un SQLite real

`database/schema.sql` y `database/sample_data.sql` se cargaron con `sqlite3` (módulo de Python) sin errores, y se ejecutaron sobre ellos las consultas de progreso documentadas. Recuentos obtenidos:

Tabla	Filas
resources	20
characters	9
missions	14
scenarios	42
badges	11
collection_items	24
glossary	13

`sample_data.sql` no está escrito a mano: lo genera `tools/generar_sample_data.py` leyendo los ficheros Kotlin de `data/seed`, así que no puede desviarse del código.

4.3 Integridad del contenido

Verificado con script sobre el código fuente:

- 42 escenarios definidos, 42 referenciados por misiones, **0 huérfanos y 0 referencias rotas** en ambos sentidos.
- 14 misiones, encadenadas m01 → m14.
- Reparto de mecánicas: ESPECIALIZACION 9, INTERCAMBIO 8, ESCASEZ 6, COOPERACION 5, EVALUAR_OFERTA 4, DECISION 4, COSTO_OPORTUNIDAD 3, CADENA 3. **Solo el 9,5 % del contenido es «valorar una propuesta»**, muy por debajo del máximo del 50 % exigido.
- Longitud de textos medida sobre los 42 escenarios: situación máx. 153 caracteres, instrucción máx. 78, explicación final mín. 81. Todos dentro de los límites que comprueban las pruebas.

4.4 Revisión estática de referencias

Se recorrieron los 87 ficheros Kotlin con un analizador propio que compara los identificadores usados contra los importados y los declarados en el mismo paquete. **No apareció ninguna referencia sin resolver** en las capas de interfaz (que es donde suelen faltar importaciones de Compose). Todos los avisos restantes eran falsos positivos conocidos del analizador (entradas de `enum`, constantes de `companion object` y miembros del mismo fichero).

Se comprobó también que no hay redeclaraciones de nombres de nivel superior dentro de un mismo paquete.

Esto no sustituye a una compilación. Un analizador de texto no detecta errores de tipos, de firmas ni de resolución de sobrecargas.

4.5 Iconos del lanzador

`tools/generate_launcher_icons.py` se ejecutó y generó los diez PNG (`mipmap-mdpi` a `mipmap-xxxhdpi`, normal y redondo). Se inspeccionó visualmente el resultado de 144×144: se ve el valle, el camino, la manzana, el pan y las dos flechas del intercambio.

5. Métricas del proyecto (contadas, no estimadas)

Métrica	Valor
Ficheros Kotlin de aplicación	75
Líneas de aplicación	~12 100
Ficheros Kotlin de prueba	12
Líneas de prueba	~1 610
Pruebas JVM escritas	136
Pruebas instrumentadas escritas	1
Tablas Room	22
Motores del dominio	11
Pantallas	11
Escenarios	42 en 14 misiones
Ilustraciones dibujadas con Canvas	20 recursos + 9 personajes + 8 avatares + 13 edificios + 11 insignias + 13 conceptos + paisaje
Permisos declarados en el manifiesto	0

6. Cómo obtener el APK

Opción A — GitHub Actions (incluida en el proyecto)

Sube el proyecto a un repositorio de GitHub. El flujo `.github/workflows/build.yml` se ejecuta al hacer push a `main` o `master` (o a mano desde la pestaña *Actions*) y hace:

```
clean → testDebugUnitTest → lintDebug → assembleDebug
```

Publica dos artefactos: `RedEconomica-v1.0.0-apk` (con el APK y su SHA-256) e `informes` (resultados de pruebas y lint en HTML).

No se ha creado ningún repositorio ni se ha hecho ningún push desde aquí.

Opción B — Android Studio

Abre la carpeta con Android Studio (Koala o posterior), deja que sincronice Gradle y ejecuta `Build → Build APK(s)`, o desde consola:

```
./gradlew clean testDebugUnitTest lintDebug assembleDebug
```

Requisitos: JDK 17 y SDK de la plataforma 34.

7. Qué esperar en la primera compilación

Esta es la primera vez que este código pasa por un compilador. Es razonable que aparezcan errores menores de la clase que un analizador de texto no ve (importaciones de Compose que falten, alguna firma cambiada entre versiones de Material 3). El diseño reduce el riesgo —el dominio es Kotlin puro y sin dependencias, y ahí está la lógica delicada— pero conviene reservar un rato para el primer `sync` de Gradle.

Si algo falla, el orden útil es: primero `testDebugUnitTest` (solo dominio y datos, sin interfaz), y después `assembleDebug`.

8. Entregables

```
deliverables/
  RedEconomica-v1.0.0-source.zip      código fuente completo
  MEMORIA_DESCRIPTIVA.pdf
  MANUAL_USUARIO.pdf
  MANUAL_TECNICO.pdf
  SHA256SUMS.txt                     huellas SHA-256 de los cuatro anteriores
```

No hay `RedEconomica-v1.0.0.apk`, por el motivo explicado en el apartado 2. El APK se obtiene con cualquiera de las dos opciones del apartado 6.

Las huellas SHA-256 están en `deliverables/SHA256SUMS.txt` y no dentro de este informe, porque este informe forma parte del ZIP: incluir aquí la huella del ZIP sería imposible (cambiaría al escribirla) y la de los PDF sería falsa en cuanto se regeneraran. El fichero de huellas se calcula al final, sobre los archivos que se entregan.

Los cinco documentos en PDF (los tres anteriores más `BASE_DE_DATOS.pdf` y este mismo informe) están además en `docs/pdf/` dentro del ZIP.

9. Declaración de honestidad

Ningún dato de este informe está inventado. Las tareas de Gradle **no se ejecutaron** y por eso no se informa de ningún resultado de pruebas, ningún tamaño de APK y ningún SHA-256 de APK. Lo que aparece en el apartado 4 se ejecutó de verdad en este entorno y se transcribe tal cual.